

Project 1 - Reinforcement Learning

Šimon Gavula

1 Introduction

This document contains the results for Homework Assignment 1 on Reinforcement Learning. The purpose of this assignment was to implement and compare two different algorithms (Temporal Difference Learning and SARSA) within a Blackjack environment.

2 Answers to the questions

2.1 Propose three possible nontrivial reasonable ways how to define the state in the game.

1. (s_p, s_d) where s_p is the sum of player's cards and s_d is the dealer's upcard.
2. (s_p, s_d, a) where s_p is the sum of player's cards, s_d is the dealer's upcard, and a is a boolean indicating a usable ace.
3. (s_p, n_p, s_d, a) where s_p is the sum of player's cards, n_p is the count of player's cards, s_d is the dealer's upcard, and a is a boolean indicating a usable ace.

2.2 For each state-space representation from 1, provide a rough estimate of the overall number of states.

For each representation, the total number of states $|S|$ is calculated by multiplying the cardinality of each individual variable's set of possible values.

1. **State:** (s_p, s_d)
Player sum $s_p \in \{4, \dots, 21\}$ (18 values) and dealer's upcard $s_d \in \{2, \dots, 11\}$ (10 values).

$$|S| = 18 \times 10 = 180 \text{ states}$$

2. **State:** (s_p, s_d, a)
To calculate size of this state space we must account for the fact that a *usable* ace is only possible for certain player's sums.

- **Sums 4–11:** For these sums, a usable ace is impossible (e.g., the lowest sum with an Ace counted as 11 is $A + A$, which starts the usable range at 12).

$$\text{States}_{4-11} = 8 \text{ (sums)} \times 10 \text{ (dealer's cards)} \times 1 \text{ (no usable ace)} = 80$$

- **Sums 12–21:** For these sums, the ace can be usable or non-usable ($a \in \{T, F\}$).

$$\text{States}_{12-21} = 10 \text{ (sums)} \times 10 \text{ (dealer's cards)} \times 2 \text{ (usable/non-usable)} = 200$$

$$|S| = 80 + 200 = 280 \text{ states}$$

3. **State:** (s_p, n_p, s_d, a)

This representation includes the number of cards in player's hand n_p . It varies depending on the player's sum s_p . For any given sum, the minimum number of cards is 2, and the maximum

number of cards $\max(n_p)$ is determined by the combination of the smallest possible card values that reach that sum without busting.

We multiply the number of valid counts for each specific sum by the 10 possible dealer's cards and the usable ace factor (where applicable).

- **Calculation Logic:**

$$|S| = \sum_{s_p=4}^{21} (\text{count of valid } n_p \text{ for } s_p) \times 10 \text{ (dealer's cards)} \times (\text{usable ace factor})$$

- **Example for Sum 10:** Minimum cards = 2 (5+5). Maximum cards = 7 (A, A, A, A, 2, 2, 2). Valid counts $n_p \in \{2, 3, 4, 5, 6, 7\}$ (6 values).

$$\text{States for sum 10} = 6 \times 10 \times 1 = 60 \text{ states}$$

- **Example for Sum 21:** Minimum cards = 2 (10+A). Maximum cards = 7 (A, A, A, A, 2, 2, 2, 3, 3, 3). Valid counts $n_p \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$ (10 values).

$$\text{States for sum 21} = 10 \times 10 \times 2 = 200 \text{ states}$$

- **Final State Space Calculation**

By summing the number of valid card counts (n_p) for every possible player sum from 4 to 21, we identify 193 unique combinations of (player's sum, card count, usable ace). Multiplying this by the 10 possible dealer's upcards gives the final total:

$$|S| = 193 \times 10 = 1930 \text{ states}$$

Note: The state count for the third representation (1920) is a high estimate. To simplify the math, we allowed combinations like {A, A, 2} to count as a sum of 4. However, the environment always counts the first Ace as 11 if the total stays below 21. In a real game, {A, A, 2} would be a soft 14, not a 4.

2.3 Pick one of the state space representations you proposed in 1. Use this representation from now on. Please, explain why you consider it the best one.

We chose the state representation: (s_p, s_d, a) because it is the optimal way to keep the state space relatively small, but to have enough information to learn smart moves. By saving Usable Ace information, the agent knows the difference between a hard 17 (where it might bust if it hits) and a soft 17 (where it is safe to hit). If we did not include the Ace, the agent would make more mistakes. This setup gives the agent enough information to make good choices without making the state space too large to slow the learning process.

2.4 Does this representation capture all information that can be used for agent decisions? Or is there any simplification? If yes, will the simplification influence the result (final policy, utility values)? If yes, how much will the result be influenced?

This representation is a simplification because it only looks at the total sum and ignores the exact cards in the player's hand. For example, the agent treats a 16 made of two cards (10 + 6) the same as a 16 made of four cards (5 + 4 + 4 + 3). This has influence on final policy because if a player has a 16 made of many small cards, those small cards are no longer in the deck and there is a larger probability that he draws the next card with higher value and possibly bust.

2.5 Can you use exact methods (value iteration/policy iteration) to solve the game? If yes, how? If not, why?

Yes, it is possible to use model-based methods like Value Iteration or Policy Iteration, but we would first need to calculate all transition probabilities such as the exact chance of drawing a 7 when the player already has a 10 and a 4. However, this approach is computationally more complex. On the other hand, in Model-Free methods like SARSA the agent learns transition probabilities implicitly through its own experience playing the game.

2.6 Briefly explain your choice of discount factor and the number of games that you need to learn the Q-values.

We found that Blackjack is an episodic game that ends quickly, meaning long-term planning over many steps is not required. Therefore, the discount factor should be close to 1. In our training, we set it to 0.99. To find the optimal learning duration, we tested a range of episodes between 100 000 and 2 000 000. We selected 1 500 000 episodes, as this provided the most consistent and accurate results.

2.7 What is the agent's expected or average utility?

Strategy	Average Utility
Random Strategy	-0.392525
Dealer's Strategy	-0.079679
TD Learning	-0.079250
SARSA	-0.047692

Table 1: Comparison of Average Utility across different agents.

From the table we can see that the random agent performed the worst with a utility of -0.39 . The dealer and TD-Learning strategies yielded nearly identical results (-0.079 and -0.079 respectively), confirming that the TD agent successfully converged to the value of the fixed dealer's policy. The SARSA agent achieved the highest utility at -0.047 which demonstrates that the agent learned strategy that outperforms the standard dealer's rules.

2.8 How fast do algorithms implemented in 4 and 5 learn?

TD-Learning (Task 4): Since this algorithm evaluates a fixed strategy (the dealer's rules), it learns quickly (it does not need to explore different actions). In our implementation, the agent required only 100 000 episodes to converge to the same average utility as the dealer's policy.

SARSA (Task 5): This algorithm takes longer to train because it must actively explore the state-action space. In our case, to find the optimal moves for each state and successfully outperform the dealer's strategy, the SARSA agent required 1 500 000 episodes.

2.9 Does the learned utility contradict your intuition?

Our results match what we expected: the SARSA agent should play better than both a random strategy and the dealer's strategy. The results confirm this, as the SARSA utility (-0.047) is higher than the random (-0.39) and the dealer's utility (-0.079). The values in the Q -table also follow the basic logic of Blackjack:

- **Player Sum 21:** The agent gave standing a very high score (around 0.9), meaning it is almost certain to win. Hitting on 21 has a negative score because it leads to an immediate bust.
- **Soft vs. hard score:** The agent learned that hitting is a much better choice with a high *soft* total (with an Ace) than with a high *hard* total (no Ace). This matches our intuition that a usable Ace protects the player from busting, allowing to play more optimally.

Table 2: Q-values from the SARSA agent's Q-table - soft vs hard score.

Player Hand	Dealer's Upcard	Usable Ace	Q(Stand)	Q(Hit)	Best Action
17 (Hard)	3	No	-0.1657	-0.3138	STAND
17 (Soft)	3	Yes	-0.045	0.0065	HIT

2.10 What is the utility for drawing a card when you have club nine, diamond jack and spades two in your hand, and dealer has club four?

The utility for drawing a card (hitting) in this situation is negative (-0.0297). This is much lower than the utility for standing, which is 0.8797 . The total of the player's hands is 21 ($9 + 10 + 2$). Since 21 is the maximum score in Blackjack, any card I draw will cause me to bust immediately. Our agent successfully learned that the better action in this case is to stand.

2.11 What is the utility of the situation when you have diamond ace and spades five and dealer spades ace? Is it better to draw a card in this situation or not?

In this situation, the utility of drawing a card is -0.4244 , compared to -0.4438 for standing. It is better to draw because the usable ace provides protection, which our agent successfully learned.

Table 3: Q-values from the SARSA agent's Q-table.

Player Hand	Dealer's Upcard	Usable Ace	Q(Stand)	Q(Hit)	Best Action
16 (Soft)	A	Yes	-0.4438	-0.4244	HIT
21 (Hard)	4	No	0.8298	-0.0297	STAND

2.12 Did your utility values estimate/ Q values converge? Did they converge to the true state values, i.e., U and Q?

TD-Learning: Yes. The average utility of the TD learning agent (-0.0792) is almost identical to the fixed dealer's strategy (-0.0796), which is expected because both use the same policy. This confirms that the TD algorithm correctly calculated the true value of U.

SARSA: To check if the agent's learning converged, we looked at the moving average of the rewards. Although the line does not show a traditional learning pattern with a steep beginning followed by a flat line, the changes are very small. The rewards stay between -0.04 and -0.06 for the entire training time. This shows that the agent converged to the optimal policy.

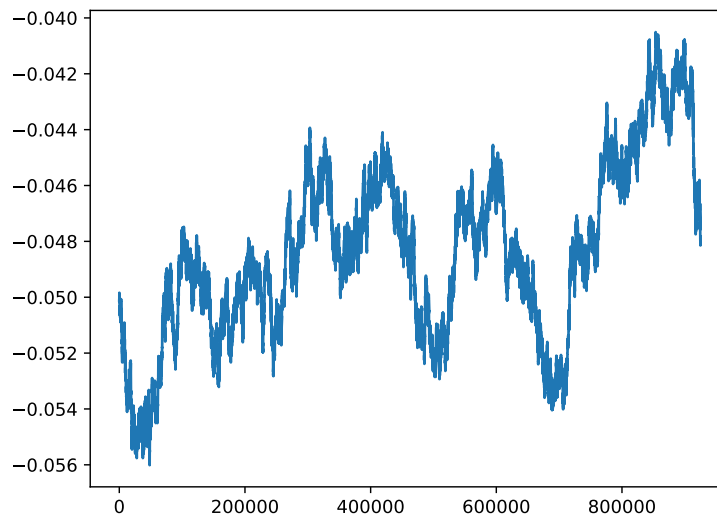


Figure 1: Moving average of SARSA rewards for 1 500 000 episodes.

2.13 Does the strategy learned by SARSA follow the recommendation in the section Blackjack strategy of?

We compared the strategy learned by the SARSA agent with the standard Blackjack strategy found on Wikipedia.

- **Hard Totals:** The strategies are very similar. Both the agent and Wikipedia recommend standing (red) on high totals (17–21) and hitting (green) on low totals. Furthermore, the agent correctly identified the dealer’s bust zone, where it stays on lower numbers (like 13–16) when the dealer shows a weak card (2–6).

Player hand	Dealer's face-up card										
	2	3	4	5	6	7	8	9	10	A	
Hard totals (excluding pairs)											
18–21	S	S	S	S	S	S	S	S	S	S	S
17	S	S	S	S	S	S	S	S	S	S	Us
16	S	S	S	S	S	H	H	Uh	Uh	Uh	Us
15	S	S	S	S	S	H	H	H	Uh	Uh	Us
13–14	S	S	S	S	S	H	H	H	H	H	H
12	H	H	S	S	S	H	H	H	H	H	H
11	Dh	Dh	Dh	Dh	Dh	Dh	Dh	Dh	Dh	Dh	Dh
10	Dh	Dh	Dh	Dh	Dh	Dh	Dh	Dh	H	H	H
9	H	Dh	Dh	Dh	Dh	H	H	H	H	H	H
5–8	H	H	H	H	H	H	H	H	H	H	H

Figure 2: Wikipedia: Hard Totals

Hard Totals											
Dealer's Upcard											
	2	3	4	5	6	7	8	9	10	11	
Player's Hand	21	S	S	S	S	S	S	S	S	S	
	20	S	S	S	S	S	S	S	S	S	
	19	S	S	S	S	S	S	S	S	S	
	18	S	S	S	S	S	S	S	S	S	
	17	S	S	S	S	S	S	S	S	S	
	16	S	S	S	S	S	H	H	H	H	
	15	S	S	S	S	S	H	H	H	H	
	14	S	H	S	S	S	H	H	H	H	
	13	H	H	S	S	S	H	H	H	H	
	12	H	H	H	H	H	H	H	H	H	
	11	H	H	H	H	H	H	H	H	H	
	10	H	H	H	H	H	H	H	H	H	
	9	H	H	H	H	H	H	H	H	H	
	8	H	H	H	H	H	H	H	H	H	
	7	H	H	H	H	H	H	H	H	H	
	6	H	H	H	H	H	H	H	H	H	
5	H	H	H	H	H	H	H	H	H		

Figure 3: SARSA Agent: Hard Totals

- **Soft Totals:** This area shows more differences. While the agent learned the general rule that hitting is safer with a usable Ace, its strategy is more aggressive (more green) than Wikipedia's. Wikipedia recommends standing on Soft 18, 19, and 20, whereas the agent chooses to hit more often at these values.

Soft totals											
	2	3	4	5	6	7	8	9	10	A	
A,9	S	S	S	S	S	S	S	S	S	S	S
A,8	S	S	S	S	Ds	S	S	S	S	S	S
A,7	Ds	Ds	Ds	Ds	Ds	S	S	H	H	H	H
A,6	H	Dh	Dh	Dh	Dh	H	H	H	H	H	H
A,4–A,5	H	H	Dh	Dh	Dh	H	H	H	H	H	H
A,2–A,3	H	H	H	Dh	Dh	H	H	H	H	H	H
A,A ^[a]	H	H	H	H	Dh	H	H	H	H	H	H

Figure 4: Wikipedia: Soft Totals

Soft Totals

		Dealer's Upcard									
		2	3	4	5	6	7	8	9	10	11
Player's Hand	21	S	S	S	S	S	S	S	S	S	S
	20	S	S	S	S	S	S	S	S	S	S
	19	H	S	S	H	S	H	S	S	S	S
	18	S	S	H	H	H	H	S	H	H	H
	17	H	H	H	H	H	H	H	H	H	H
	16	H	H	H	H	H	H	H	H	H	H
	15	H	H	H	H	H	H	H	H	H	H
	14	H	H	H	H	H	H	H	H	H	H
	13	H	H	H	H	H	H	H	H	H	H
	12	H	H	H	H	H	H	H	H	H	H

Figure 5: SARSA Agent: Soft Totals